

Technical Specification: Non-Destructive MQ Header Auditor (Module: MQ-PEEK-AUDIT)

Attribute	Details
Component Name	MQ Header Correlation Validator
Target Manager	QM_Evoke_Platform
Category	Distributed Event Orchestration / Observability
Status	Draft / Specification

1. System Overview

The **MQ Header Auditor** is a specialized testbed utility designed to validate transaction integrity without interfering with the production message flow. In standard MQ operations, a "Get" operation is destructive. This module utilizes the **MQ Browse** cursor to inspect the **MQMD** (Message Descriptor) of messages residing on the queue, specifically to verify that the **Correlation_ID** matches upstream records in **tbl_LocalMessages**.

2. Technical Requirements Mapping

Requirement ID	Definition	Implementation Strategy
ORCH-2.1.1	Non-destructive MQ Peeking	Initialize MQOPEN with MQOO_BROWSE access.
ORCH-2.1.2	Correlation_ID Extraction	Parse the MQMD.CorrelId byte array into a Hex/String format.

ORCH-2.1.3	SQL Chaining Validation	Execute lookup against tbl_LocalMessages using the extracted ID.
ORCH-2.1.4	Smart Polling	Implement MQGMO_WAIT with a WaitInterval of 300,000ms.

3. Component Architecture

3.1 MQ Connection & Access Logic

To ensure non-destructive behavior, the connection handle must be opened with specific options that prevent the "locking" or "removal" of messages.

- **Open Options:** MQOO_BROWSE | MQOO_FAIL_IF QUIESCING
- **Get Message Options (GMO):**
 - MQGMO_BROWSE_FIRST: To start at the head of the queue.
 - MQGMO_BROWSE_NEXT: To iterate if multiple messages are present.
 - MQGMO_WAIT: To enable the smart polling window.

3.2 Header Parser (The MQMD Buffer)

The auditor focuses exclusively on the **Message Descriptor (MQMD)**. It should not load the message payload into memory unless specifically required, optimizing performance.

- **Target Field:** MQMD.CorrelId (24-byte binary field).
- **Normalization:** Convert the binary CorrelId to a UTF-8 string or Hexadecimal string to match the data type stored in the SQL backend.

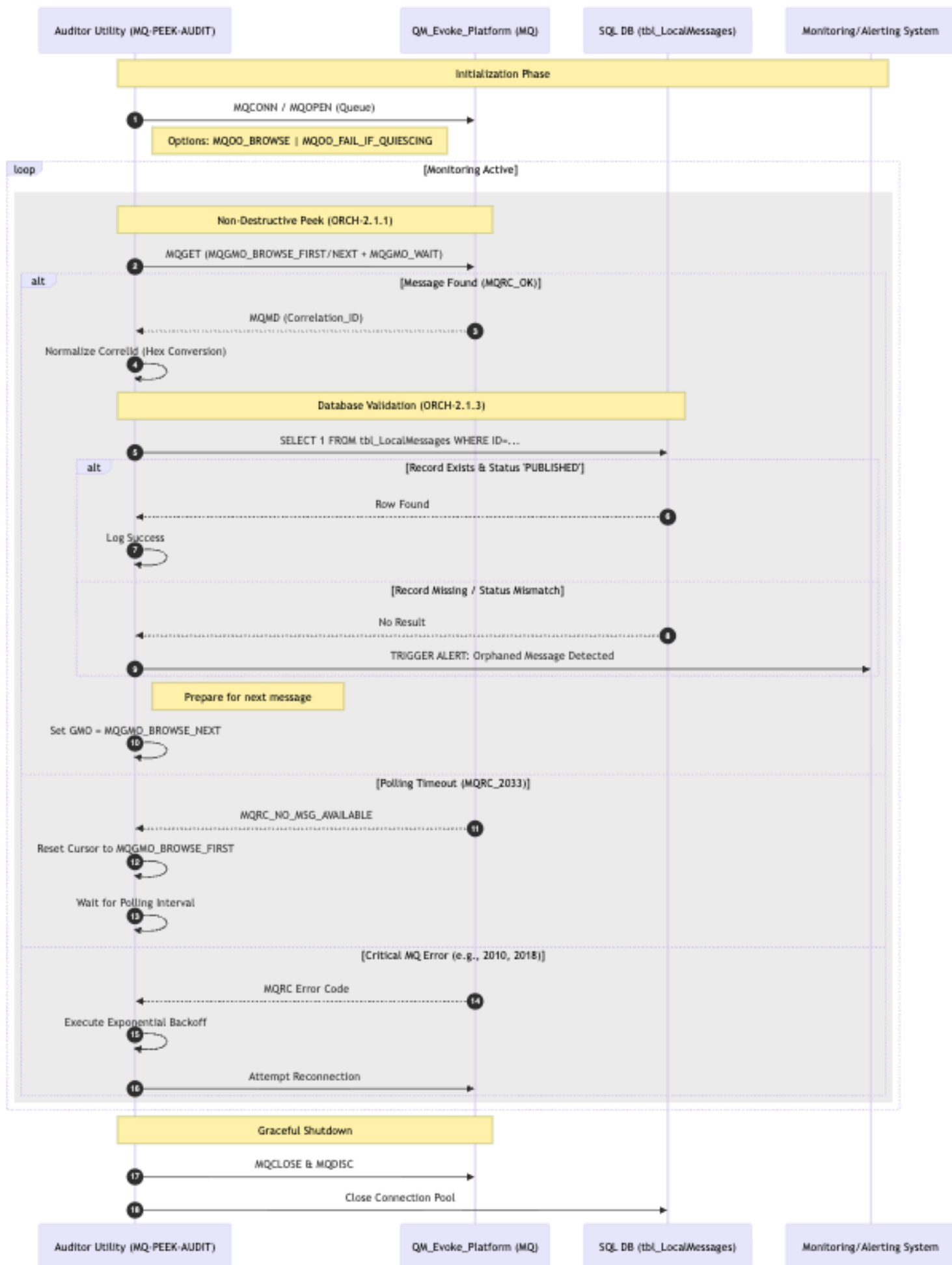
3.3 Database Validation Layer

The validator performs a passive check against the local orchestration table.

SQL

None

```
-- ORCH-2.1.3 Logic
SELECT 1
FROM tbl_LocalMessages
WHERE Correlation_ID = :extracted_id
      AND Status = 'PUBLISHED';
```



4. Operational Logic (Pseudo-Code)

Python

```
None

# Initialize MQ Connection
queue = mq.Queue(queue_manager, "TARGET_QUEUE", MQOO_BROWSE)

gmo = mq.GMO()
gmo.Options = MQGMO_BROWSE_FIRST | MQGMO_WAIT
gmo.WaitInterval = 300000 # 300 Seconds (ORCH-2.1.4)

while monitoring_active:
    try:
        # Peek at the message
        message = queue.get(None, None, gmo)

        # Extract ID (ORCH-2.1.2)
        correl_id = message.correlId.hex()

        # Cross-Reference (ORCH-2.1.3)
        exists = db.query("SELECT COUNT(*) FROM tbl_LocalMessages WHERE ID=?", correl_id)

        if exists:
            logger.info(f"Success: Correlation ID {correl_id} validated.")
        else:
            logger.error(f"Alert: Orphaned message detected. ID {correl_id} missing in DB.")

        # Move cursor to next message without removing current
        gmo.Options = MQGMO_BROWSE_NEXT

    except MQError as e:
        if e.reason == MQRC_NO_MSG_AVAILABLE:
            continue # Wait for next polling window
```

5. Success Metrics & KPIs

- **Integrity:** 100% extraction accuracy. The MQMD must be read in its entirety.
- **Non-Interference:** The queue depth must remain unchanged by the Auditor. Use of MQGMO_MSG_UNDER_CURSOR or standard MQGET is strictly prohibited.

- **Concurrency:** The Auditor must allow the primary consumer application to perform destructive MQGET operations simultaneously without causing 2042 (Object in use) errors.
-

6. Error Handling

- **MQRC_NO_MSG_AVAILABLE (2033):** Expected behavior during polling. System should reset the WaitInterval.
- **MQRC_NOT_AUTHORIZED (2035):** Occurs if MQOO_BROWSE permissions are not granted to the Service Account.
- **Truncation:** Ensure the buffer size is sufficient to read the MQMD even if the payload is large (though payload is ignored).

Would you like me to generate a sample unit test suite in Java or Python to validate this "Browse" logic?